SYSTEM ANALYSIS & DESIGN

1.Problem Statement & Objectives

1.1. Problem Statement

Communication between deaf and hearing individuals can be challenging due to the communication gap caused by differences in language and communication methods.

Traditional communication often requires someone who understands sign language to interpret the signs. This may not always be available, which can limit independent communication.

EchoSign addresses this challenge by developing a Machine Learning-based sign language recognition system that analyzes visual sign data and predicts the corresponding Arabic words.

The system uses extracted features from hand, face, and body landmarks to train a machine learning model capable of recognizing sign patterns and classifying them into their corresponding Arabic words.

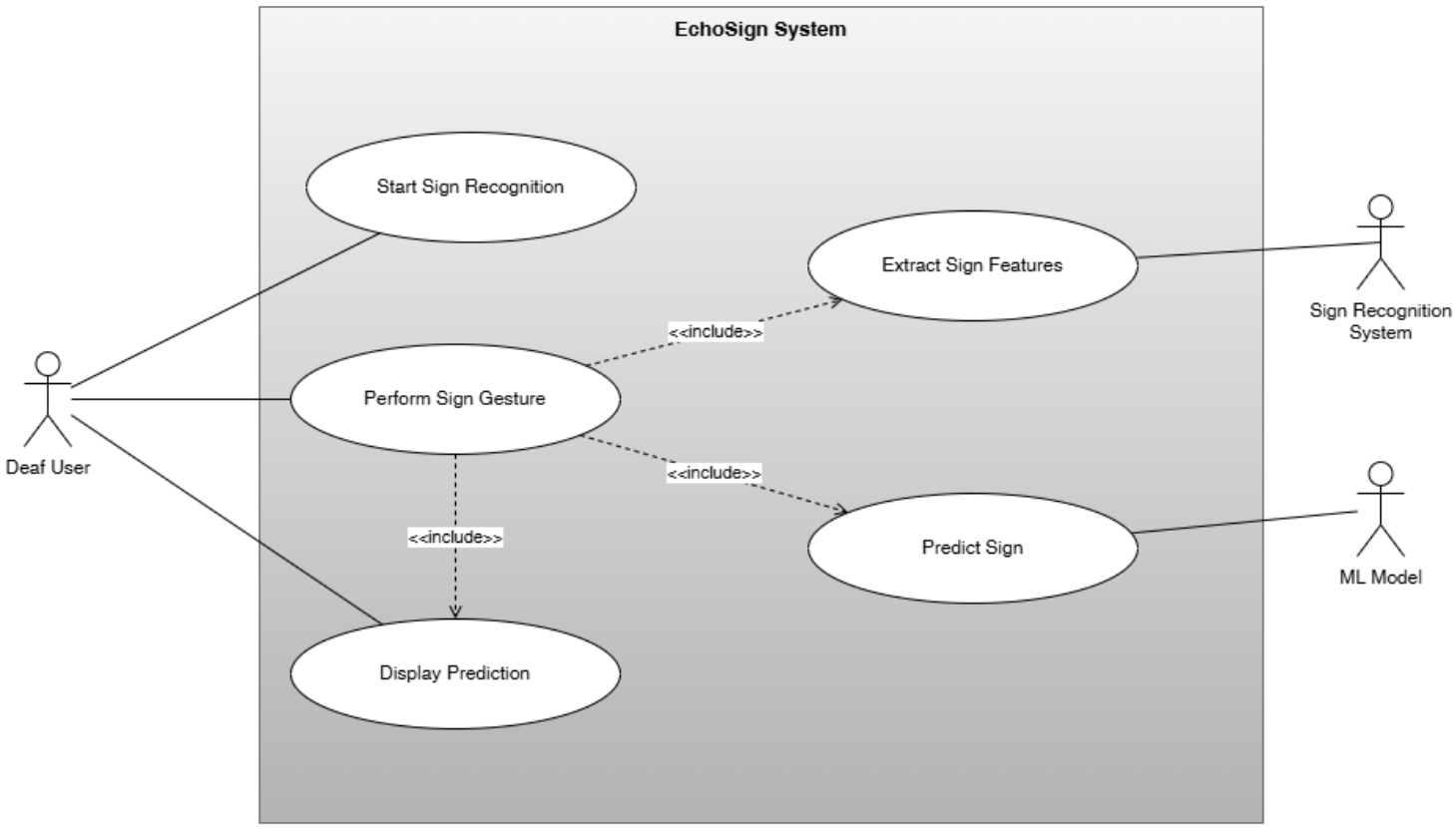
The main challenge is to build a reliable recognition model that can learn meaningful patterns from sign language data and provide accurate predictions when users perform signs in front of a camera.

1.2.Project Objectives

The main objectives of EchoSign are:

1. **Sign Language Recognition:** Develop a Machine Learning model capable of recognizing Arabic sign language gestures.
2. **Feature Extraction:** Utilize hand, face, and body landmarks to represent sign language movements as structured numerical features.
3. **Data Preprocessing:** Prepare and organize the extracted landmark data to make it suitable for model training.
4. **Model Training:** Train and evaluate machine learning and deep learning models to identify the most suitable approach for sign recognition.
5. **Sign Classification:** Classify input sign sequences into their corresponding Arabic words.
6. **Real-Time Prediction:** Integrate the trained model with a mobile application to display predicted Arabic words while users perform signs in front of the camera.
7. **Model Evaluation:** Evaluate model performance using appropriate classification metrics to measure its recognition capability.

1.3.Use Case Diagram



1.4.Use Case Descriptions

Use Case 1: Start Sign Recognition

Field	Description
Actor	Deaf User
Precondition	The application is open.
Trigger	The user opens the camera screen.
Main Flow	1. The user opens the Sign Recognition screen. 2. The application requests camera access if needed. 3. The camera starts capturing video.
Postcondition	The camera is ready to capture sign language gestures.

Use Case 2: Perform Sign Gesture

Field	Description
Actor	Deaf User
Precondition	The camera is active and ready.
Trigger	The user performs a sign gesture in front of the camera.
Main Flow	1. The user performs an Arabic sign. 2. The camera captures the sign. 3. The captured input is prepared for processing.
Postcondition	The sign input is ready for feature extraction.

Use Case 3: Extract Sign Features

Field	Description
Actor	Sign Recognition System
Precondition	Sign video input is available.
Trigger	The system receives captured video frames.
Main Flow	1. The system identifies hand, face, and body landmarks. 2. The landmarks are extracted. 3. The extracted landmarks are organized into structured features.
Postcondition	The extracted features are ready for preprocessing and prediction.

Use Case 4: Predict Sign

Field	Description
Actor	Trained ML Model
Precondition	The extracted features are preprocessed and ready.
Trigger	The system sends the prepared features to the trained model.
Main Flow	1. The model receives the input features. 2. The model analyzes the sign patterns. 3. The model predicts the corresponding sign class. 4. The predicted class is mapped to its Arabic word.
Postcondition	The predicted Arabic word is returned to the application.

Use Case 5: Display Prediction

Field	Description
Actor	Deaf User
Precondition	The system has received a prediction result.
Trigger	The recognition process returns an Arabic word.
Main Flow	1. The application receives the predicted word. 2. The application displays the Arabic word on the screen.
Postcondition	The user can see the recognized Arabic word.

1.5. Functional Requirements

ID	Functional Requirement
FR-01	The system shall provide a mobile application interface for accessing its main features.
FR-02	The system shall capture sign language gestures in real time.
FR-03	The system shall process the captured input using the sign language recognition AI model.
FR-04	The system shall communicate with the AI recognition service through an API or server, according to the implemented architecture.
FR-05	The AI model shall recognize supported Arabic sign language gestures and predict their corresponding words.
FR-06	The system shall display recognized words in Arabic on the mobile application's screen.
FR-07	The system shall continue live recognition while the translation feature is active.
FR-08	The system shall handle unrecognized gestures, network failures, and service errors appropriately.
FR-09	The system shall provide a Settings screen for the preferences supported by the application.

1.6. Non-functional Requirements

Category	Non-functional Requirement
Performance	The system should recognize gestures and display results with minimal delay to support real-time interaction.
Accuracy	The AI model should achieve an acceptable recognition accuracy for the supported sign language vocabulary, measured through testing.
Usability	The application should provide a clear, simple, and easy-to-navigate user interface.
Accessibility	Arabic text output should be readable, clearly presented, and easy to understand.
Security	Communication between the mobile application and the online recognition service should use HTTPS.
Privacy	Camera input and related data should be handled responsibly and should not be stored unnecessarily.
Reliability	The application should handle recognition errors, network interruptions, and API failures without unexpected crashes.
Compatibility	The application should operate on the Android devices and operating system versions supported by the project.
Maintainability	The mobile application, API, and AI model should be structured to facilitate testing, maintenance, and future improvements.
Availability	The online recognition service should be accessible when the service is operational and the user has an internet connection.

1.7. Software Architecture

1.7.1. Architecture Description

EchoSign follows a Client-Server Architecture, where the Flutter mobile application acts as the user interface and capture layer, while the backend server hosts the API and the Machine Learning model.

The application captures the user's sign language gestures through the camera, extracts relevant landmarks, and sends the processed data to the backend via a REST API.

The backend processes the request, uses the trained Machine Learning model to classify the sign, and returns the predicted Arabic word to the application for display.

1.7.2. System Components

1. Flutter Mobile Application

- Camera Capture: Captures the user's sign language gestures.
- Landmark Extraction: Extracts relevant hand, face, and body landmarks.
- Send Data via API: Sends the processed data to the backend server.

2. Backend API Server

- Process Input: Receives and processes incoming requests.
- Interact with ML Model: Passes the input data to the trained model.
- Return Prediction: Sends the prediction result back to the mobile application.

3. Machine Learning Model

- Sign Classification: Analyzes the extracted features and classifies the performed sign.
- Predict Arabic Word: Maps the predicted class to its corresponding Arabic word.

3. System Workflow

1. The user performs a sign language gesture in front of the mobile camera.
2. The Flutter application captures the gesture.
3. The application extracts the required landmarks.
4. The processed features are sent to the backend through a REST API request.
5. The backend passes the input data to the trained Machine Learning model.
6. The model predicts the corresponding sign class.
7. The backend maps the predicted class to its Arabic word.
8. The prediction result is returned to the Flutter application.
9. The application displays the recognized Arabic word to the user.

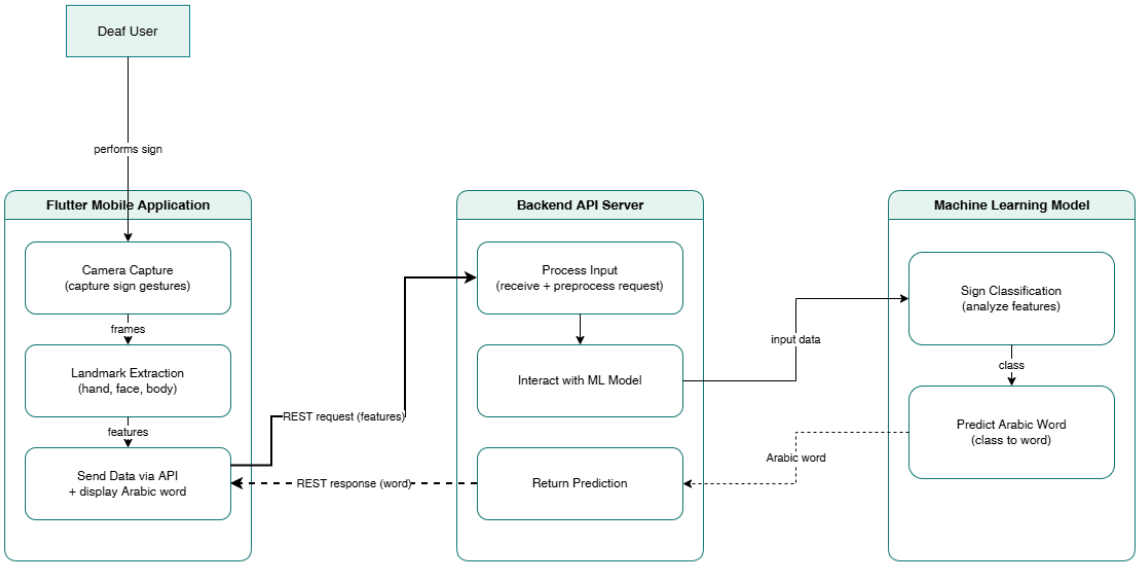
4. Architecture Style

The system uses a Client-Server Architecture with RESTful API Integration, separating the mobile application from the Machine Learning model.

This architecture supports scalability, maintainability, and efficient resource utilization by separating the user interface, backend processing, and Machine Learning inference.

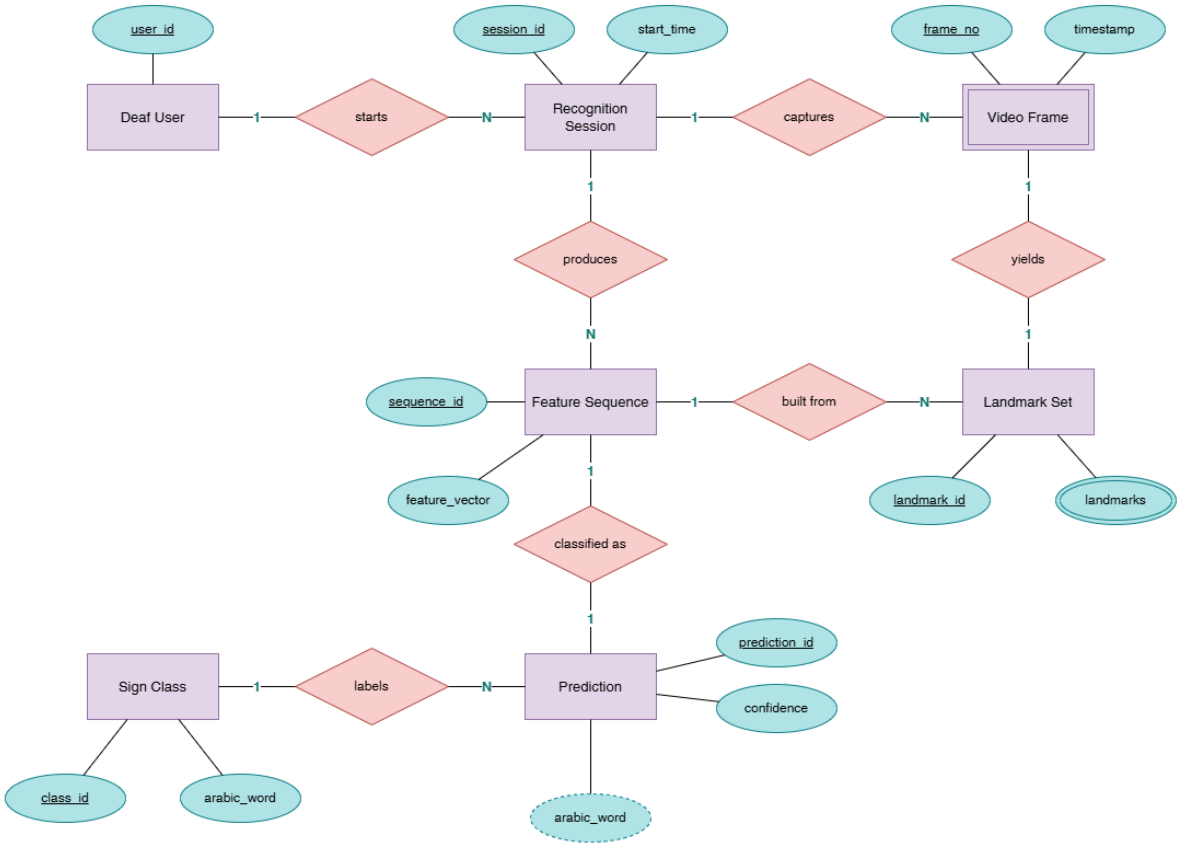
5. Key Components Summary

Component	Responsibility
Flutter Mobile Application	Handles user interaction and data capture.
Backend API	Manages requests, processes data, and integrates with the ML model.
Machine Learning Model	Performs sign classification and returns predictions.

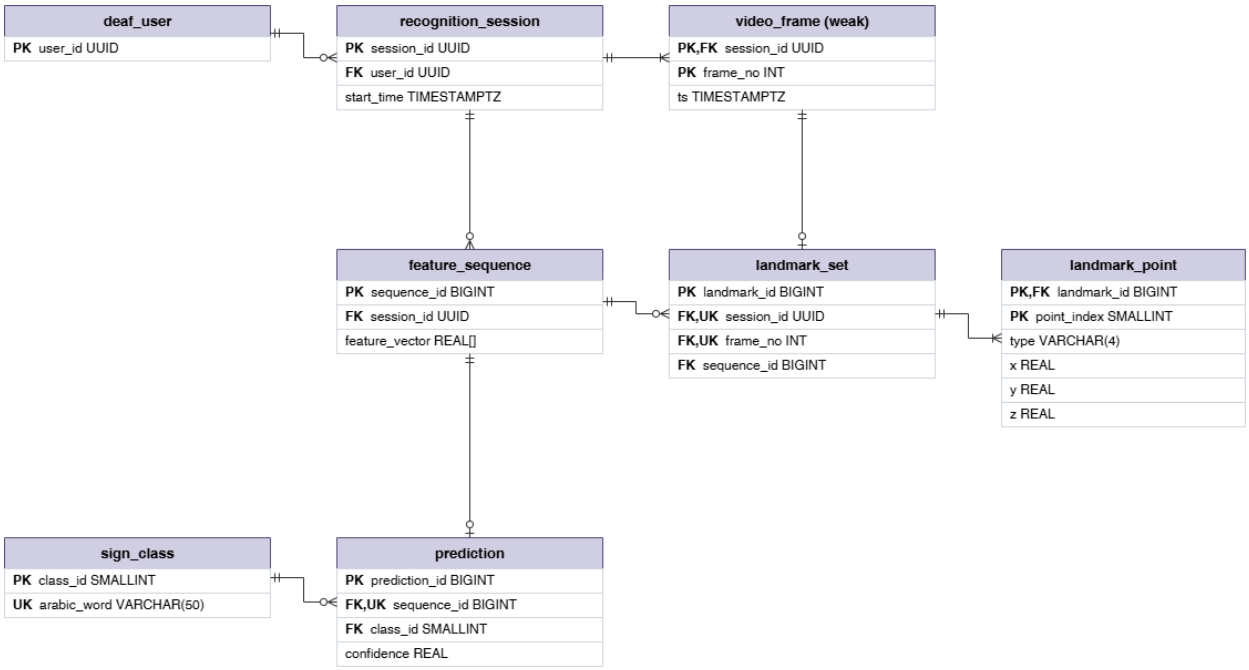


2. Database Design & Data Modeling

2.1. ER Diagram (Entity-Relationship Diagram)



2.2. Logical & Physical Schema



Logical Relationships

The conceptual model defines the following relationships:

- One user can have multiple recognition sessions.
- One recognition session can contain multiple video frames.
- One recognition session can contain multiple feature sequences.
- Each video frame can be associated with a landmark set.
- Each landmark set can contain multiple landmark points.
- A feature sequence can be associated with multiple landmark sets.
- A feature sequence can have a prediction result.
- Each sign class can be associated with multiple prediction results.

Primary keys and foreign keys are included in the conceptual diagram to illustrate entity identification and logical relationships.

These keys represent the proposed data structure and are not implemented as physical database constraints.

Physical Schema

A physical database schema is not implemented in the current version of EchoSign.

The system does not currently use a Database Management System (DBMS) or persistent relational database.

Therefore, physical table creation, SQL data types, indexes, database constraints, and SQL implementation are not applicable to the current project version.

The conceptual data model is used only to describe the logical organization of the information involved in the sign recognition process.

Normalization Considerations

Since EchoSign does not currently implement a physical relational database, database normalization has not been applied to an actual database.

However, the proposed conceptual model separates different data entities, such as users, recognition sessions, video frames, landmarks, feature sequences, sign classes, and predictions.

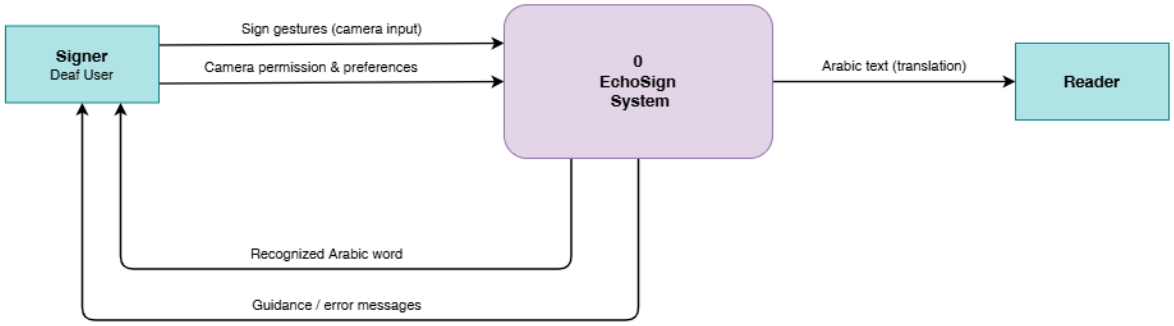
This separation helps illustrate how data can be organized independently and reduces unnecessary duplication in a possible future database implementation.

Normalization principles such as First Normal Form (1NF), Second Normal Form (2NF), and Third Normal Form (3NF) may be considered if a relational database is introduced in future versions.

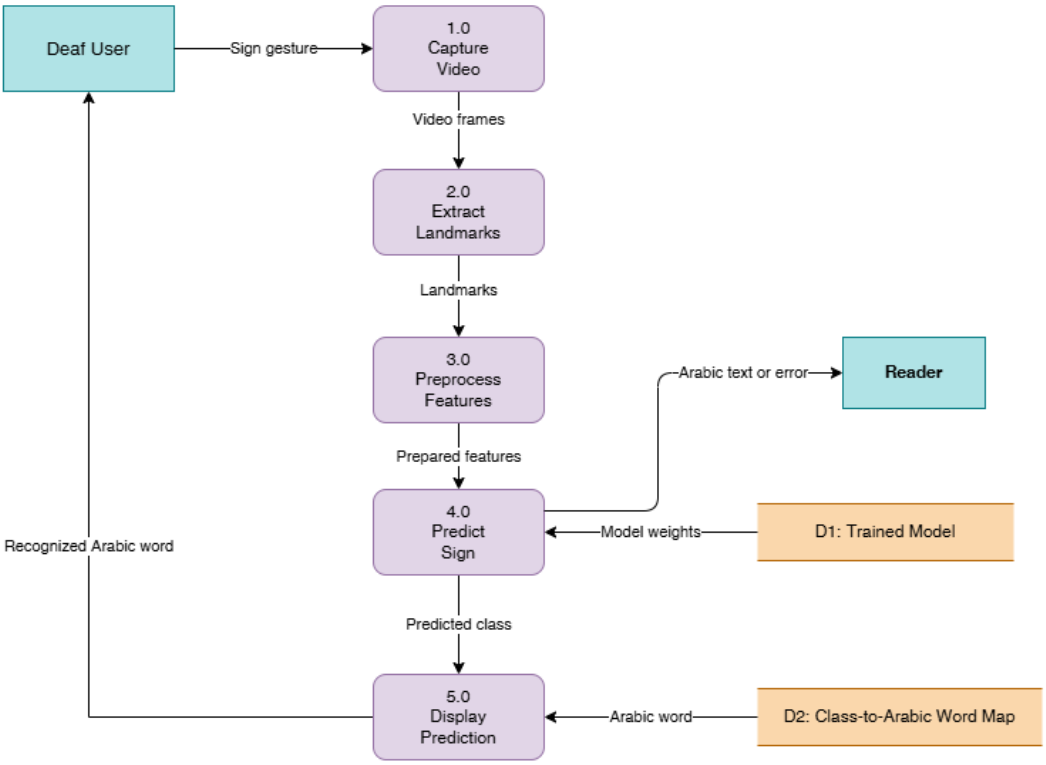
3. Data Flow & System Behavior

3.1.DFD (Data Flow Diagram)

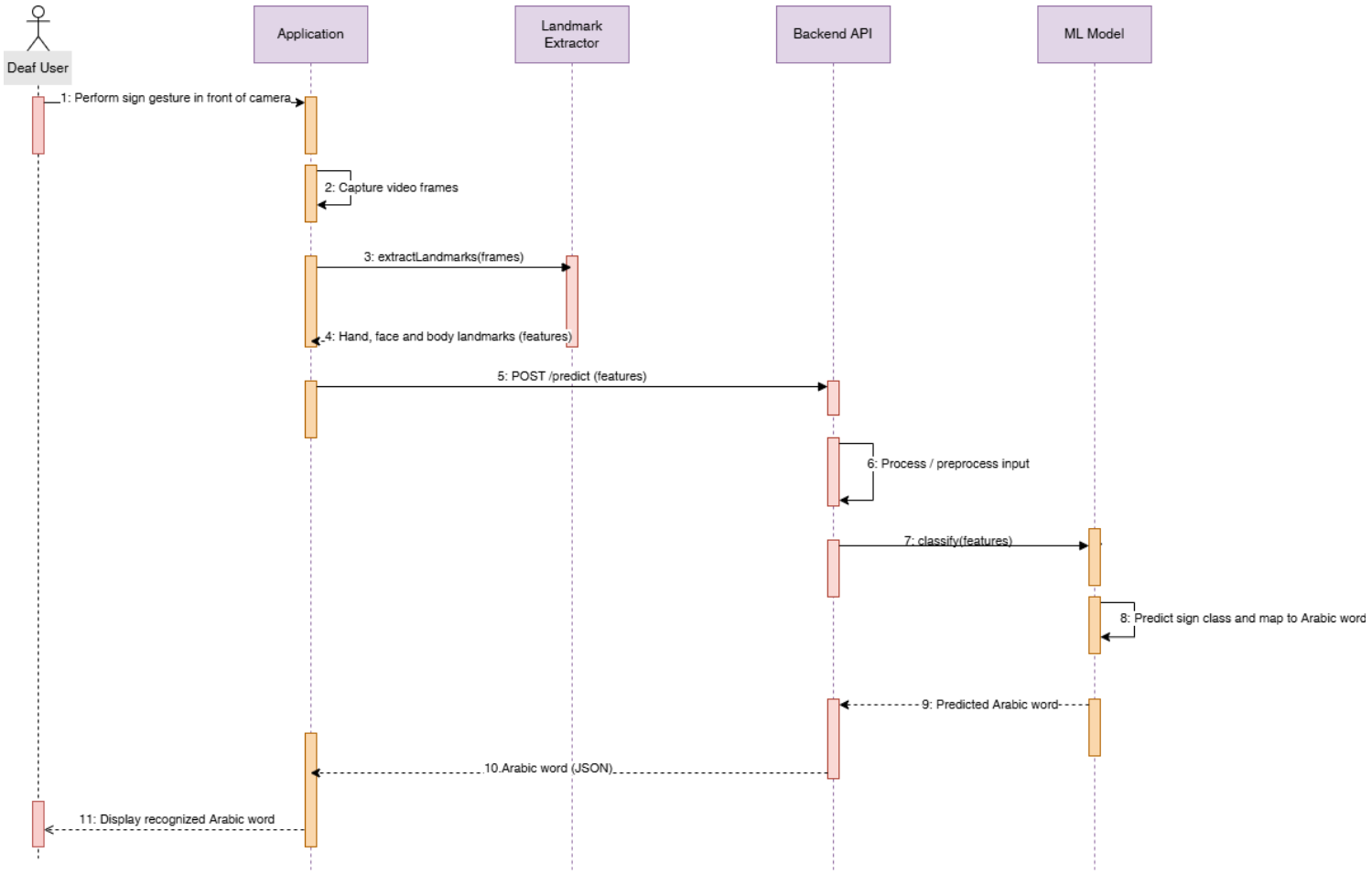
3.1.1.Context Diagram (level 0)



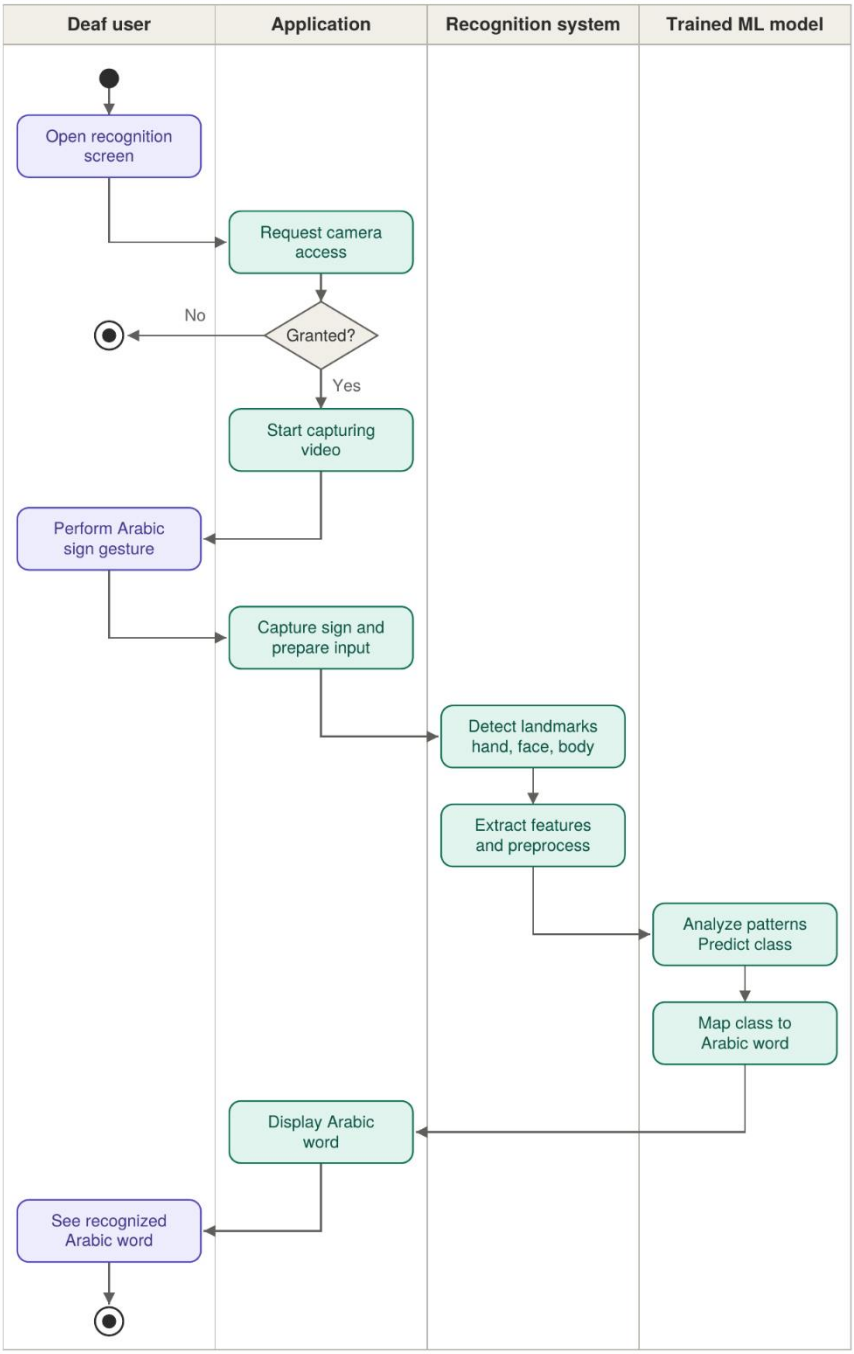
3.1.2.DFD (level 1)



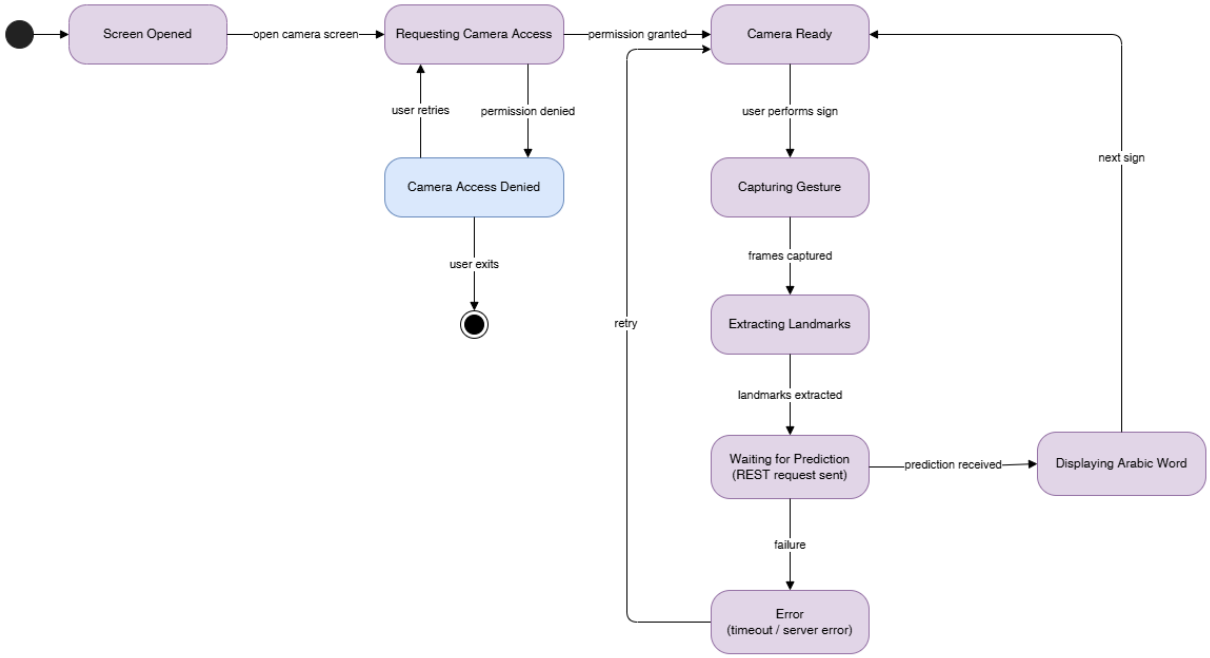
3.2.Sequence Diagrams



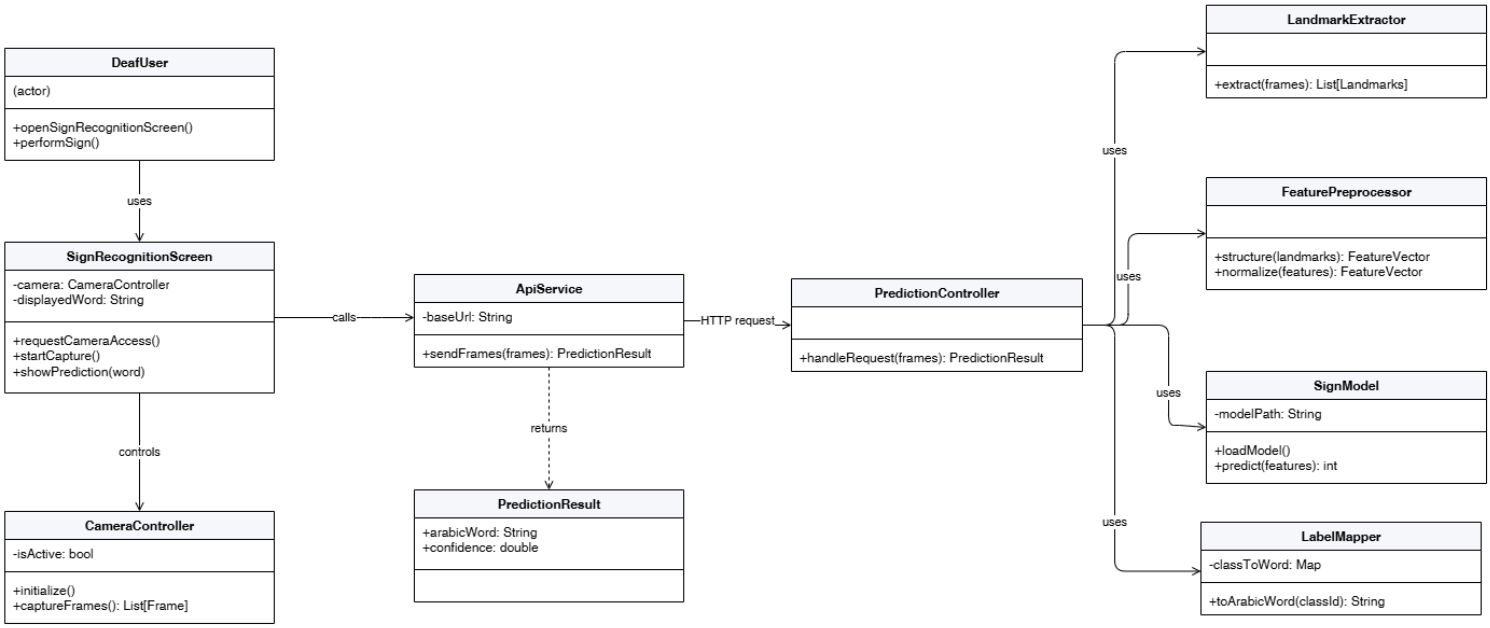
3.3. Activity Diagram



3.4. State Diagram

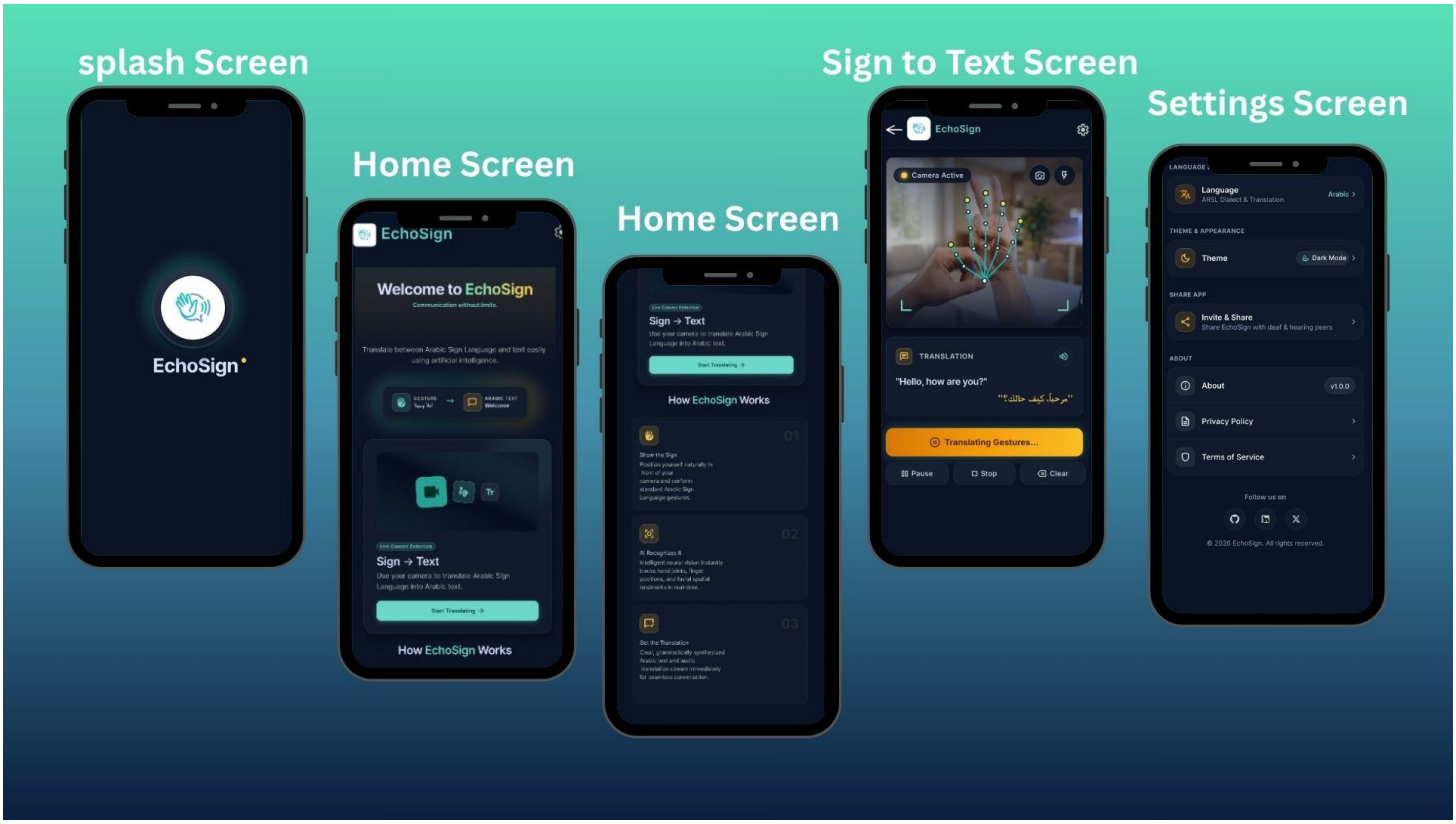


3.5. Class Diagram



4. UI/UX Design & Prototyping

4.1. Wireframes & Mockups



4.2. UI/UX Overview & Guidelines

4.2.1 Screens Overview & Flow

The application features 4 meticulously crafted screens designed in Figma to deliver a seamless, distraction-free user experience:

1. Splash Screen:
    - A clean and elegant introductory screen displaying the application's logo and visual identity.
2. Home / Onboarding Screen:
    - Features a welcome introduction card explaining the core value proposition: *"Translate between Arabic Sign Language and text easily using artificial intelligence."*
    - A prominent teal call-to-action button to start live translation (Start Translating ->).
    - A 3-step guide detailing how the system works: *(01. Show the Sign →02. AI Recognizes It →03. Get the Translation)*.
3. Live Recognition Screen (Camera & Translation):
    - Camera Preview: Displays real-time video streaming with hand landmark tracking visualization (Camera Active indicator).
    - Translation Card: Shows the translated text in English and Arabic (*"Hello, how are you?"* / *"مرحباً، كيف حالك؟"*) inside a dedicated container, complete with text-to-speech capabilities.
    - Control Bar: Features an active status indicator (Translating Gestures...) alongside intuitive control actions (Pause, Stop, Clear).
4. Settings Screen:
    - Provides configuration options for language exchange (Arabic Sign Language dialecst), appearance themes (Dark Mode), app sharing, privacy policy, and terms of service.

4.2.2. UI/UX Guidelines

1. Design Principles

- Real-time Focus: Prioritizing an unobstructed camera viewport while ensuring the translated output is prominently highlighted in the lower focal area.
- Minimalistic & Modern: Utilizing rounded cards and buttons (Rounded Corners: 16px–24px) to create a modern, approachable interface.

2. Color Palette

- Primary Background: Dark Navy / Slate (#0B132B / #0F172A) – A dark, eye-friendly backdrop that conserves battery life during continuous camera sessions.
- Accent / Action Color: Cyan / Teal (#48CAE4 / #50E3C2) – Used for primary actions, active highlights, and landmarks tracking mesh.
- Secondary Accent: Warm Gold / Yellow (#FFC107) – Applied to active state indicators and status prompts (Translating Gestures...).
- Card Surface: Dark Slate Blue (#1E293B) – Used for container backgrounds to establish visual depth and contrast.

3. Typography

- Arabic Font: *Cairo* / *Tajawal* – Bold and highly legible weights optimized for reading translated text quickly from a distance.
- English Font: *Inter* / *Plus Jakarta Sans* – Clean, geometric sans-serif fonts suited for UI labels and headings.

4. Accessibility Considerations (Deaf & Hard of Hearing Users)

- High Contrast Text: High-contrast typography (white/yellow text on dark slate surfaces) ensures clear legibility under various lighting conditions.
- Visual Status Indicators: Live mesh tracking points overlaying the user's hands provide immediate feedback that the AI is tracking gestures successfully.
- Large Touch Targets: Generous padding and sizing on interactive elements (Start Translating, Pause, Stop) allow for comfortable, single-handed interaction.

5. System Deployment & Integration

5.1 Technology Stack

EchoSign uses a Machine Learning-based technology stack to develop an Arabic Sign Language Recognition mobile application.

The system consists of a frontend mobile application, a backend API, and a Machine Learning model responsible for sign classification.

The main technologies and their responsibilities are described below:

1. Frontend Technologies

- Flutter: Used to develop the cross-platform mobile application.
- Dart: Used as the primary programming language for the mobile application.

2. Backend Technologies

- Backend API: Responsible for receiving requests from the Flutter application and integrating with the trained Machine Learning model .
- Backend Framework: FastAPI.

3. Machine Learning Technologies

- Python: Used for Machine Learning model development and data processing.
- Machine Learning Libraries & Framework : Pandas , Numpy, MediaPipe, PyTorch, Scikit-Learn.

4. Database Technologies

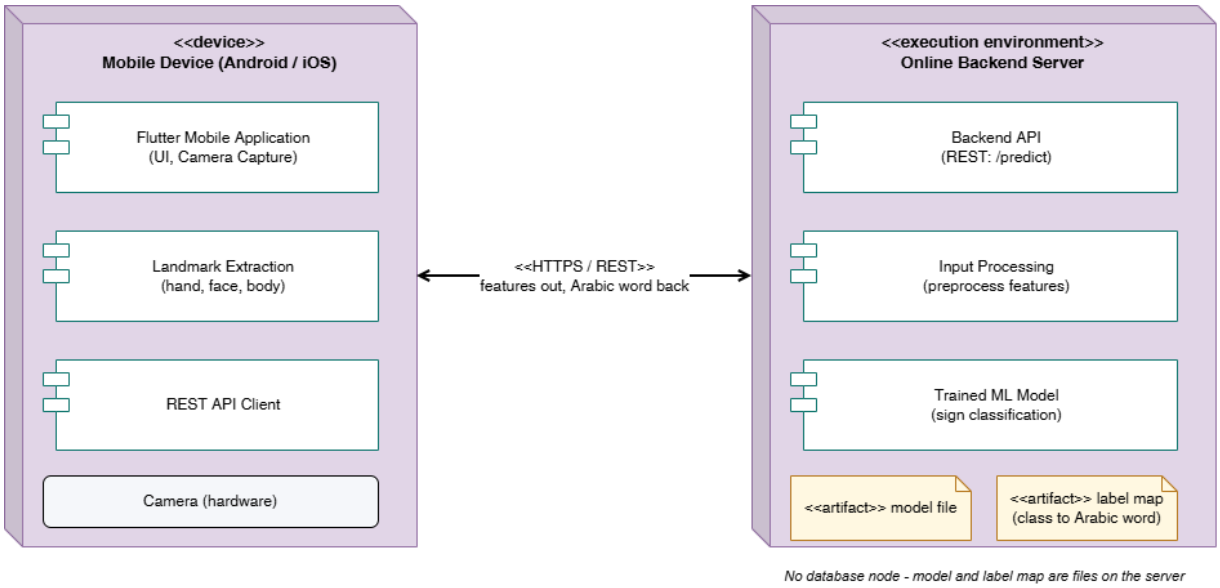
- No database management system is currently implemented.
- The current system does not maintain persistent relational database records.

5. Communication Technologies

- REST API: Used for communication between the Flutter application and the backend recognition service.

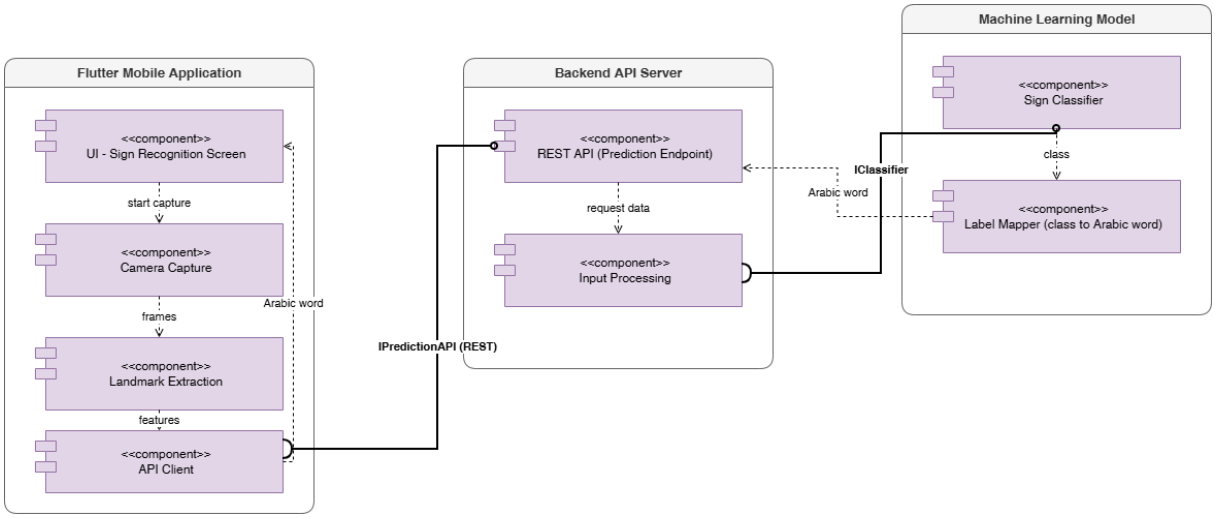
Layer	Technology	Purpose
Frontend	Flutter	Mobile application
Programming Language	Dart	Frontend development
Backend	FastAPI	API handling
ML Development	Python	Model development
ML Framework	PyTorch	Model training and inference
Communication	REST API	Frontend-backend communication
Database	Not implemented	No persistent database

5.2 Deployment Diagram



No database node - model and label map are files on the server

5.3.Component Diagram



6. Additional Deliverables

6.1 API Documentation

API Documentation

EchoSign uses API-based communication to connect the Flutter mobile application with the backend Machine Learning recognition service.

The API acts as an interface between the mobile application and the trained Machine Learning model.

It is responsible for receiving the extracted sign language features, processing the recognition request, and returning the predicted Arabic word.

The API documentation describes the available endpoints, HTTP methods, request parameters, response formats, and error handling mechanisms.

Proposed Recognition Endpoint

The following endpoint represents the proposed sign recognition operation. Its final URL and request structure will be confirmed during backend implementation.

API Endpoint Specification

Field	Description
Endpoint	/predict (proposed)
HTTP Method	POST
Purpose	Recognize an Arabic sign
Input	Extracted landmark features
Content Type	application/json (proposed)
Output	Predicted sign class and Arabic word
Response	JSON (proposed)

6.2 Testing & Validation

Testing and validation are important to ensure that the EchoSign system performs correctly and provides reliable Arabic Sign Language Recognition.

The testing process covers the Machine Learning model, backend API, and Flutter mobile application.

1. Machine Learning Model Testing

The trained model will be evaluated using the following metrics:

- Accuracy: Measures the overall percentage of correct predictions.
- Precision: Measures how many predicted instances of a class are correct.
- Recall: Measures how many actual instances of a class are correctly identified.
- F1-Score: Measures the balance between precision and recall.
- Confusion Matrix: Evaluates classification performance across different sign classes.

2. Unit Testing

Individual system components will be tested separately, including:

- Landmark extraction and feature preparation.
- Model prediction functionality.
- Prediction-to-Arabic-word mapping.
- API request and response handling.

3. Integration Testing

Integration testing will verify that the main system components work together correctly.

- Verify communication between Flutter and the backend API.
- Verify that the backend receives the expected input features.
- Verify that the trained model processes the received input.
- Verify that prediction results are returned correctly.
- Verify that the mobile application displays the returned Arabic word.

4. User Acceptance Testing (UAT)

The application will be evaluated by intended users to assess:

- Ease of use and navigation.
- Camera functionality.
- Clarity of displayed Arabic predictions.
- Overall recognition experience.

Testing results will be documented to identify errors, evaluate performance, and improve system reliability.

6.3 Deployment Strategy

EchoSign follows a client-server deployment approach that separates the mobile application from the backend Machine Learning recognition service.

6.3.1. Mobile Application Deployment

The Flutter application will be built as an Android APK and installed on compatible mobile devices.

6.3.2. Backend API Deployment

The backend API will be deployed to a suitable hosting environment to receive requests from the mobile application and manage communication with the Machine Learning model.

6.3.3. Machine Learning Model Deployment

The trained Machine Learning model will be made available to the backend service for inference. It will process incoming sign features and generate the corresponding prediction.

6.3.4. Communication Security

The mobile application will communicate with the backend through secure HTTPS requests when deployed.

6.3.5. Model Updates

The Machine Learning model can be updated independently of the mobile application, allowing future improvements to recognition performance.

6.3.6. Scalability Considerations

The backend architecture can be extended in the future to support more users, additional sign classes, and increased prediction requests.

No database deployment is required in the current version of EchoSign.